

To: Charalampos Mainas (@cmainas) & Anastassios Nanos (@ananos) — Nubificus / urunc

14 May 2026

Project: CNCF · urunc · Improve lifecycle management of sandbox monitors

Upstream: urunc-dev/urunc#112

1. About me

I am Raman Kapoor — a B.Tech (Information Technology) graduate from ABV-IIITM Gwalior (class of 2026), currently working at Juspay, India's largest payments orchestration platform, on low-level Linux infrastructure. My most recent piece there was leading the migration of our block-storage tier from CEPH to Lightbits NVMe/TCP — same workload at 48K TPS versus 37K TPS on roughly a tenth of the hardware (four nodes instead of forty), with active-active Barbican HA on a shared backend. Before that I shipped a thread-local and server-local cache layer in Rust on our hot APIs that cut cross-node calls by ~4% at peak, and a Go-based Shamir-secrets distribution broker on top of OpenBao that removed our reliance on AWS KMS for key delivery.

2. What I have been working on

2.1 Open-source contributions to Namma Yatri (Haskell, production)

The work I am proudest of, and the closest match to what this LFX project will involve in terms of *shipping upstream code in a large unfamiliar codebase with maintainer review*, is my contribution to [Namma Yatri](#) — the open-source ride-hailing platform that runs in production across India. Between November 2025 and January 2026 I shipped three merged PRs into the org:

- **feat** [nammayatri/nammayatri#13442](#) — Special-zone driver-payout scheduling system. A distributed, event-driven scheduler serving roughly 10,000 rides per day; cut driver payout latency from weekly cycles down to T+2 hours under production traffic. The interesting part was the Redis-backed delayed-job scheduler with per-ride locks and idempotent state transitions — exactly-once execution without adding latency to the ride-booking hot path.
- **feat** [nammayatri/nammayatri#13452](#) — Exposed the resulting driver-payout state to the driver-app frontend through the API layer.
- **feat** [nammayatri/shared-kernel#1108](#) — PaytmEDC payment-flow integration in the shared kernel — config management, checksum generation, API interaction. Went through several review iterations before merge.

All three are merged and live. Working on a Haskell codebase with no prior Haskell production experience, taking serious code-review, and getting to "merged" three times in a row, is the muscle I expect this LFX project to need.

2.2 Juspay — production infrastructure work

Beyond the storage migration mentioned above, the work at Juspay most relevant to this project is the secrets-distribution layer: a Go-based broker on top of OpenBao that uses Shamir secret-sharing for

key distribution, keeps keys memory-only, and replicates via Raft. The integration includes RS256 token signing and IMDS-based identity discovery for secure VM bootstrap. That body of work is where most of my `syscall`-heavy Go and Unix-socket experience comes from — the exact surface this LFX project lives on.

2.3 Independent project: PostgreSQL on Kubernetes / OpenStack

A 5-layer nested virtualization stack (KVM → OpenStack → Kubernetes → CloudNativePG → Postgres) on a single i9-14900, optimized via NUMA-aware CPU pinning and 16 GB hugepages. Zero-touch HA provisioning using Cluster API (CAPO) and CloudNativePG — ~12 min cluster bring-up, ~7 s automatic failover, RPO=0. A separate Go-based OpenBao broker plus a custom Python SDK (`osvault`) handles secret delivery to VM bootstrap. Not directly urunc, but the same shape of problem: API-driven lifecycle of stateful infrastructure on top of VMM-based isolation.

2.4 BREAK OS — a 32-bit i386 kernel from scratch

A small monolithic kernel I wrote in C++ from scratch — bootstrap, protected mode, PIC + ISR/IRQ wiring, 64 KB GDT, ~0.1 ms interrupt response. I include it because it is where my interest in monitors and hypervisors started, and because the LFX project is, in some sense, the userspace side of the same world.

2.5 urunc so far — what I have learned through issue #389

A few weeks ago I picked up [urunc-dev/urunc#389](#) — high host CPU caused by console output in running containers. A prior contributor had attributed the regression to `bufio.Scanner` in `cmd/urunc/log_forward.go`. My own profiling showed that path is not hot during guest execution: that scanner only handles the parent-child JSON log pipe during `urunc create`; once `syscall.Exec()` runs, the VMM manages stdio directly. The real cost lives one layer deeper.

I wrote a small KVM-exit and throughput sampler — `bench-console.sh`, which ananos has since indicated they would like to fold into urunc's CI as a regression check — and traced the regression to per-byte PIO traps on the 16550 UART at port `0x3F8`. The `-serial stdio` baseline costs roughly 1.16 M PIO exits per 5-second window, with QEMU's `address_space_write` dominating the host flame graph. A simple `virtio-console` swap drops PIO exits to ~0 and lifts throughput from 2,016 to 73,378 lines/sec (about 36×), closing the runc-vs-urunc gap on this workload from roughly 407× down to ~11×.

The thread has since covered: how to differentiate `virtio-console` from `serial` without breaking Linux kernels built without `CONFIG_VIRTIO_CONSOLE` (my first proposal, a capability-intersection design borrowed from urunc's rootfs path, was constructively rejected by @cmainas for exactly that reason); the upcoming pivot from the `com.urunc.unikernel.cmdline` annotation to a `kernelArgs`-style annotation that lets users specify boot parameters at build time; and an invitation from @ananos to draft a blog post on the investigation for the Nubificus / CNCF blog.

Through that work I have gained working familiarity with:

- urunc's monitor invocation path — `pkg/unikontainers/hypervisors/qemu.go`, `BuildExecCmd`, and the `syscall.Exec`-style handoff in `unikontainers.go`.
- The distinction between urunc's parent-child JSON log pipe (consumed by `bufio.Scanner` before `syscall.Exec`) and the VMM-managed stdio path that takes over after.

- The QEMU + KVM monitor surface — what costs PIO exits at the 16550 UART, how virtio devices change the picture, where the cost actually lives.
- urunc's Linux unikernel path — `pkg/unikontainers/unikernels/linux.go`, the boot-params construction, the urunit handshake.
- The `VMM` interface in `pkg/unikontainers/types/types.go` and how each monitor (QEMU, Firecracker, HVT) implements it today.
- An empirical debugging posture for urunc — `/sys/kernel/debug/kvm/io_exits`, `perf` flame graphs, and a small reproducer image, rather than guessing.

3. The project — monitor lifecycle (urunc#112)

3.1 The problem

urunc today shells out to QEMU or Firecracker with a long `-x ... -y ...` command string. That is enough to *start* a sandbox, but everything after that — checking what the guest is doing, hotplugging a device, asking the monitor to shut down gracefully, attaching a console after the fact — is either impossible or has to be approximated by signalling the host process and hoping for the best. Both QEMU and Firecracker already expose a much better interface for this: QEMU has the QEMU Machine Protocol (QMP, line-delimited JSON over a Unix socket), and Firecracker exposes a REST-style API socket. Issue [#112](#) is exactly this gap — urunc has CLI for spawn but no socket-based control plane for the lifecycle that follows.

The cost of not having that surface shows up in real places. Graceful pause/resume is not really possible. Hotplug is not really possible. The urunc-side state machine has to infer monitor state from process exit codes instead of asking the monitor directly. And each new monitor we add (cloud-hypervisor, Dragonball, etc.) is a fresh integration with no shared abstraction.

3.2 My approach

I see this project as two pieces that have to land together. The first is a **monitor-agnostic control interface inside urunc** — an internal Go interface that captures the operations urunc actually needs (probe, query state, send shutdown, hotplug net/block, attach console). The existing `VMM` interface is the natural home; today it carries `BuildExecCmd`, `Stop`, `Path`, `UsesKVM`, `SupportsSharedfs`, `Ok`, and `PreExec` for HVT seccomp. The new surface extends it without breaking those. The second piece is the **per-monitor implementations** that translate the interface into QMP for QEMU and into the Firecracker API for Firecracker, plus a small fallback for monitors that do not expose a socket (so the existing CLI-only flow keeps working for HVT and similar).

For QEMU, I would generate `-qmp unix:<sock>,server,nowait` as part of the existing `BuildExecCmd` flow. After spawn, the sequence is: read the greeting, send `qmp_capabilities`, then any of the operational commands the interface exposes. The QMP commands I would implement in order of project priority are `query-status` (probe + state machine), `quit` (graceful shutdown, replaces `killProcess`), `stop` / `cont` (pause/resume), `device_add` / `device_del` (hotplug for virtio-net and virtio-blk first), `chardev-add` / `chardev-remove` (console attach/detach), and `query-vcpus` / `query-block` for introspection.

For Firecracker, the existing `firecracker-go-sdk` already wraps the API socket; the work is mostly wiring `urunc`'s interface methods to `PauseVM` / `ResumeVM` / `PatchDrive` / `PatchMMDS` / `GetMachineConfig`, and deciding which subset we actually expose at the `urunc` layer. Firecracker's API surface is smaller than QEMU's, so the interface needs capability flags — the same shape `urunc` already uses for `SupportsSharedfs`.

3.3 Decision points the design doc will resolve

I have a working position on each of these, but I expect the design doc and your review to settle them — these are not unilateral decisions and I want to be explicit that I know that. On a related note: I have already been through one design-discussion round with @cmainas on this codebase. My first proposal for the `virtio-vs-serial` console choice (capability-intersection, borrowed from `urunc`'s `rootfs` path) was constructively rejected because Linux kernels can be built without `CONFIG_VIRTIO_CONSOLE`. The feedback shape from that round is what I expect from this project's design-doc review, and I have re-set my priors accordingly.

Decision 1 · QMP wire types — hand-roll vs adopt a library

Either declare our own Go structs for the small subset of QMP messages we use, or pull in github.com/digitalocean/go-qemu (or similar). Default position: hand-roll, because the subset is small and the third-party libraries pull in a lot of code we will not use. Open to evidence otherwise.

Decision 2 · Extend `VMM` vs introduce a sibling `MonitorControl` interface

Either grow the existing `VMM` interface with the new methods, or keep `VMM` as-is (spawn/exec concerns) and add a parallel `MonitorControl` interface for post-spawn operations. Default position: sibling interface, because spawn and runtime-control have different lifetimes (the runtime interface only becomes valid after the socket handshake succeeds).

Decision 3 · Socket location and cleanup ownership

Per-container subdirectory under `urunc`'s state dir, removed on container delete. Default position: yes, but who owns the cleanup if `urunc` crashes between socket open and use? The fallback recovery semantics need a written policy.

Decision 4 · Opt-in vs default-on

Should the new control path be default-on for supported monitors, or opt-in via an OCI annotation initially while we burn it in? Default position: opt-in for the first cut so we do not break anyone's existing flow, default-on once the integration tests are green.

3.4 What I will explicitly not do

- I will not rip out the existing `BuildExecCmd` / `syscall.Exec` shape. The new socket-based control is layered on top, not a replacement.
- I will not also rewrite the `seccomp` handling (`PreExec` for HVT) or the monitor-`rootfs` work in the same project. Those are adjacent but big enough to be their own conversations.
- I will not add a new monitor (cloud-hypervisor, Dragonball, etc.) inside the LFX scope. The interface should make adding one later easier — that is a different person's first PR.
- I will not introduce a new dependency for QMP unless the design doc justifies it.

3.5 What I will prove out before merging

- The QMP-controlled QEMU path can spawn, query state, gracefully shut down, and hotplug a virtio-net device end-to-end, with the existing nginx-on-unikraft integration tests still green.
- The Firecracker path can do the same subset.
- A simple benchmark — same workload as my #389 work, but measuring monitor-control overhead (round-trip count, socket I/O cost). The design doc gets a real number, not just "it works."
- The fallback path (CLI-only, no socket) still works for every monitor that uses it today.

4. Plan — 12 weeks (8 Jun – 31 Aug 2026)

I plan to break the implementation phases into multiple small PRs rather than one large one, so review can happen in parallel with development. Weekly check-ins with the mentors plus an open draft PR for the design doc as soon as it is started.

WK	PHASE	CONCRETE DELIVERABLE
1–2	Onboarding + design doc, draft 1	Re-read <code>pkg/unikontainers/hypervisors/</code> . Author a "current state of monitor lifecycle in urunc" gist for mentors to correct. Open the draft design doc PR proposing the <code>MonitorControl</code> interface, QMP + Firecracker mapping, socket lifecycle, and resolving decisions 1–4.
3	Design doc revisions, sign-off	Address review comments. Tag the doc as ready once mentors sign off. The interface signature and dependency decision are settled before any production code lands.
4–5	QEMU QMP — MVP then full lifecycle	Wire <code>-qmp</code> into QEMU <code>BuildExecCmd</code> . Implement capability handshake, <code>query-status</code> , <code>quit</code> first (small PR). Follow with <code>cont / stop</code> , <code>device_add / device_del</code> for virtio-net and virtio-blk, error handling for unsupported commands. Unit tests for the wire codec.
6–7	Firecracker API + state-machine integration	Implement the same <code>MonitorControl</code> surface against <code>firecracker-go-sdk</code> — pause/resume, drive patch, network patch where applicable. Plumb the control path through <code>unikontainers.go</code> so urunc's state machine uses it. Confirm the CLI-only fallback still works.
8–9	Tests, CI, hotplug demo	Integration tests in the existing layout. Extend <code>bench-console.sh</code> to time QMP round-trip cost. CI workflow update. End-to-end demo: start a unikraft-nginx sandbox, hotplug a second virtio-net device, verify connectivity, document the recipe.
10	Bug-fix + user-facing docs	Review-comment cleanup across all open PRs. User docs: how to talk to a urunc sandbox's QMP socket, how to opt into the new control path, what guarantees the interface gives.
11–12	Buffer + final clean-up + blog post	Reserved for slippage. If none, take a follow-up issue from the mentors' backlog. Mentee blog post for the CNCF / Nubificus blog covering the architecture and the numbers.

5. Pre-application work — in flight before May 19

Between now and the application deadline I am landing the following in the proposal repo (github.com/unchangedraman/proposal) so you can see the work, not just read the plan:

- A small standalone Go program that opens a Unix socket to a `qemu-system-x86_64` instance, sends `qmp_capabilities`, then `query-status` and `quit`. ~100 lines, intentionally throwaway — just to prove the wire shape and surface the rough edges.
- A second, equally small Go program against the Firecracker API socket using `firecracker-go-sdk`, doing `GetMachineConfig` and `PauseVM` / `ResumeVM` on a stopped microVM.
- A one-page sketch (`docs/design-sketch.md`) of what the `MonitorControl` interface signature would look like and where it slots into `pkg/unikontainers/hypervisors/types.go`. Not a finished design — a starting point for the actual design doc in weeks 1–2.

This is not part of the LFX commitment; it is what I am doing this week regardless, because the only way to know whether the design is right is to write a little of it.